State-Filtered Deliberation: Mitigating Context Bloat for Small Open-Weight Developer Agents

Karandeep Singh Sodhi
karandeep.sodhi2504a@gmail.com
Google — The Gemma 4 Developer Agent Challenge (Paper Track) — October 2026

Abstract

Deploying autonomous software engineering developer agents on small open-weight language models (such as Google's Gemma 4 series: E2B, E4B, 12B) is constrained by context accumulation. In standard monolithic ReAct loops, raw terminal outputs, directory listings, and multi-page file reads accumulate monotonically into the context buffer. For small quantized models, this influx of task-irrelevant tokens dilutes key attention signals, elevates tool argument formatting errors, and can lead to premature context exhaustion.

To address this issue, we propose **State-Filtered Deliberation (SFD)**, a modular architecture that replaces the expanding history buffer with a sequential pipeline of three isolated sub-agents: AST Triage, Surgical Patcher, and Hermetic Verifier. At each stage boundary, SFD compresses intermediate diagnostic findings into an immutable structured YAML state packet, resetting subsequent sub-agent prompt contexts to under 1,500 tokens using Google ADK's `include_contents: "none"` directive. We provide the complete declarative system design, mathematical formulation of token signal degradation, and reproduction scripts. All prior preliminary simulated numbers have been quarantined; empirical performance metrics will be populated strictly from verified container execution traces.

1. Introduction

Autonomous software engineering (SWE) agents demonstrate promising capabilities in diagnosing bugs, reproducing failures, and synthesizing verified code patches ((Jimenez et al., 2024)). However, the majority of performant SWE systems rely on massive proprietary APIs with 100k+ token context windows. Enabling local, edge-deployable, and private software maintenance requires adapting small open-weight models—such as Google's Gemma 4 series (E2B, E4B, 12B) ((Gemma Team, 2024))—to navigate complex codebases.

On repository repair benchmarks, small models struggle significantly compared to larger models. In flat ReAct agent loops ((Yao et al., 2023)), small models frequently encounter operational hurdles:

1. **Context Window Exhaustion:** Running compiler suites or printing multi-page test tracebacks quickly exhausts the token window, terminating execution with `ContextWindowExceededError`.
2. **Attention Diffusion on Quantized Weights:** As context expands with noisy stdout traces, 4-bit quantized attention mechanisms experience dispersed attention weights, increasing tool parameter formatting errors and malformed diff headers.
3. **Harness Git Contamination:** Scratch reproduction scripts written in the repository tree pollute git diff calculations during patch extraction.

In this paper, we study the hypothesis that **unmanaged context accumulation is a primary operational failure mode of small developer agents**, and that hard context isolation between sub-goals can restore task completion reliability without modifying model weights.

We introduce **State-Filtered Deliberation (SFD)**, an architecture implemented within Google's declarative Agent Development Kit (ADK) specification. SFD decomposes the software repair task into three specialized, sequential sub-agents: (1) AST-Localized Triage, (2) Surgical Patcher, and (3) Hermetic Verifier. Sub-agents communicate strictly through an immutable YAML state packet, resetting the active prompt context at each stage boundary.

2. Context Degradation & The Token Signal Ratio

Let an autonomous agent interaction be structured across turns $t \in \{0, 1, ..., T\}$. In standard monolithic ReAct ((Yao et al., 2023)), the prompt context C_t at turn t is the historical concatenation:

$$C_t = [s_0, a_0, o_0, a_1, o_1, ..., a_{t-1}, o_{t-1}]$$

The total token sequence length $|C_t|$ expands monotonically:

$$|C_t| = |s_0| + \sum_{i=0}^{t-1}(|a_i| + |o_i|)$$

Let C_{target} denote the minimal set of code tokens strictly required to synthesize the correct patch (typically 15–35 lines of code). We define the empirical **Token Signal Ratio** $\rho(C_t)$ as:

$$\rho(C_t) \equiv \frac{|C_{\text{target}}|}{|C_t|}$$

In monolithic trajectories, tool observations o_i (e.g., compiler tracebacks, broad directory listings, multi-page file reads) introduce thousands of extraneous tokens per turn. While $|C_{\text{target}}|$ remains static once the buggy function is identified, the denominator $|C_t|$ expands at 1,500 to 2,500 tokens per interaction turn. Consequently, the signal ratio degrades rapidly as turns progress.

For small quantized models, this token dilution produces two distinct empirical failure modes:

1. **Tool-Call Formatting Failures:** As irrelevant syntax tokens proliferate in the KV cache, the probability of emitting malformed JSON delimiters or unescaped quotes increases, triggering tool rejection loops.
2. **Diff Header Corruption:** When generating unified diffs within an active context exceeding thousands of tokens, small models frequently hallucinate non-existent line offsets, causing `git apply` rejections during verification.

3. State-Filtered Deliberation Architecture

To prevent token dilution, SFD abandons append-only histories in favor of sequential sub-agent handoffs. Implemented within the declarative `adk-submission` schema, SFD deploys a `SequentialAgent` pipeline dividing the software repair lifecycle into three isolated components.

3.1 Sub-Agent 1: AST-Localized Triage (`TriageAgent`)

The Triage Agent is restricted strictly to localization and diagnostic analysis:

- **Tool Surface:** `search_similar_code`, `get_code_neighbors`, `get_code_subgraph`, and bounded `read_file` (≤ 150 lines). Arbitrary shell execution and file modifications are disabled.
- **Diagnostic Target:** The agent queries repository code intelligence graphs rather than executing broad textual sweeps (`grep -r`).
- **Output Contract:** It emits a structured, immutable YAML State Packet:

```
<triage_packet>
target_file: "fastapi/dependencies/utils.py"
target_symbol: "request_params_to_args"
line_range: [820, 845]
test_target: "tests/test_header.py -k test_underscore"
</triage_packet>
```

This packet captures the localization finding concisely.

3.2 Sub-Agent 2: Surgical Patcher (`PatchAgent`)

The core design mechanism of SFD is hard context resetting via Google ADK's configuration:

```
include_contents: "none"
```

Because `include_contents: "none"` is enforced, zero conversational history or tool observations from the triage phase are passed into the patch agent's context. The patch agent is initialized with only:

1. The original task `{problem_description}`.
2. The structured `{triage_packet}` stored in `session.state`.

This resets the active prompt context. Operating with high token signal density, the Patch Agent reads only the target line slice specified in the packet and executes localized modifications via `edit_file`.

3.3 Sub-Agent 3: Hermetic Verifier (`VerifierAgent`)

The Verifier Agent receives the localized `{triage_packet}` and `{patch_summary}`. It performs three automated verification steps:

1. **Scratch File Cleanup:** Audits `/workspace/` to ensure no temporary reproduction scripts exist (such scripts must reside in `/tmp/repro.py`), preventing untracked file inclusion during `git add -N .`.
2. **Targeted Test Execution:** Invokes `run_command` targeting strictly `test_target`. Full repository sweeps (`pytest .`) are disallowed to prevent timeout and context exhaustion.
3. **Submission Gate:** Calls `submit_patch` and validates that `files_changed > 0` before session completion.

4. System Implementation & Evaluation Protocol

4.1 Evaluation Framework

The architecture is implemented strictly within the Google ADK and `swegemma` harness specification:

1. **Phase 1 (Inference):** The agent executes inside an isolated sandbox (`Container A`) under hardware quotas (4 GB RAM, 2 vCPUs, air-gapped network).
2. **Phase 2 (Verification):** The extracted unified diff (`agent_patch`) is applied to a clean clone in `Container B` and verified against hidden tests (`test_patch`) via JUnit XML validation.

4.2 Benchmark Dataset Distribution

The official competition evaluation set comprises 129 repository repair tasks:

- `fastapi/fastapi`: 67 tasks (51.9%)
- `Textualize/rich`: 48 tasks (37.2%)
- `psf/requests`: 13 tasks (10.1%)
- `encode/httpx`: 1 task (0.8%)

4.3 Disclosure of Results Status

In accordance with scientific integrity standards, all previously drafted simulated numbers and synthetic benchmarks have been formally quarantined to `_retracted_simulation/`. The agent architecture has been deployed to the official competition pipeline (Submission Ref #57643078). Controlled empirical evaluations across baselines (flat ReAct, observation truncation, and SFD ablations) are being instrumented under real execution tracing.

5. Operational Hypotheses for Small Models

Through analysis of small model failure modes in SWE-bench-style harnesses, we formulate three operational hypotheses to be measured:

1. **Triage Mislocalization Risk:** If the initial triage phase identifies the incorrect function or file, the decoupled patcher cannot recover, as it lacks access to the full repository tree.
2. **Patcher Scope Sensitivity:** Restricting edits to minimal AST line ranges reduces syntax corruption and indentation errors on 2B–4B models.
3. **Harness Hygiene Failures:** In containerized evaluation harnesses like `swegemma`, patch extraction relies on `git add -N . && git diff`. Untracked scratch reproduction scripts in `/workspace/` pollute the patch diff unless cleaned by a verification stage.

6. Related Work

Repository-Level SWE Agents. Benchmarks like SWE-bench ((Jimenez et al., 2024)) have stimulated significant agent development. Systems like SWE-agent ((Yang et al., 2024)) introduce custom shell interfaces, while OpenHands ((Wang et al., 2024)) explores generalist coding loops. Agentless ((Xia et al., 2024)) showed that a simplified, two-phase pipeline (localize then repair) could match complex agent loops on frontier models. Our work explores a similar staged philosophy, but specifically investigates how modular state handoffs resolve context accumulation bottlenecks in small open-weight models (2B–4B).

Context Management in Agents. While Reflexion ((Shinn et al., 2023)) explores verbal memory in expanding buffers, it assumes sufficient context capacity. In resource-constrained environments, rolling windows (`EventsCompactionConfig`) retain noisy tracebacks. Hard context resets with structured state packets offer an alternative strategy for small quantized models.

7. Limitations & Threats to Validity

1. **Cascade Vulnerability:** The absence of backward history passing means that if the Triage Agent mislocalizes the fault, the downstream patcher cannot backtrack.
2. **Repository Skew:** Over 51% of benchmark tasks come from `fastapi` and 37% from `rich`. Performance may vary on languages other than Python or architectures with different test conventions.
3. **Pre-training Overlap:** As open-source repositories, portions of the benchmark codebases may have been present in pre-training corpora, though task issue descriptions reflect distinct failure states.

8. Open Resource & Reproducibility

We release the complete system artifacts under Apache 2.0:

1. **Declarative Agent Submission:** Complete `agent.yaml` package (`submission_sfd.zip`) formatted for Google ADK and `swegemma`.
2. **Live Kaggle Kernel:** Public notebook generating and packaging the agent archive.
3. **GitHub Repository:** Open-source release at `https://github.com/KarandeepSinghSodhi/gemma-sfd-agent`.

9. Conclusion

Small open-weight models require deliberate context management to avoid operational failures. State-Filtered Deliberation establishes a modular, declarative architecture decoupling localization, editing, and verification through canonical state packets.

References

Gemma Team. (2024). Gemma: Open Models Based on Gemini Research and Technology. *Google Deepmind Technical Report*.

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. (2024). SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. *International Conference on Learning Representations (ICLR)*.

Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language Agents with Verbal Reinforcement Learning. *Advances in Neural Information Processing Systems (Neurips)*.

Wang, X., Li, B., Song, Y., Frank, F. F., & others. (2024). OpenHands: An Open Platform for AI Software Developers as Generalist Agents. *Arxiv Preprint Arxiv:2407.16741*.

Xia, C. S., Deng, Y., Dunn, S., & Zhang, L. (2024). Agentless: Demystifying LLM-based Software Engineering Agents. *Arxiv Preprint Arxiv:2407.01489*.

Yang, J., Jimenez, C. E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., & Press, O. (2024). SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. *Arxiv Preprint Arxiv:2405.15793*.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. *International Conference on Learning Representations (ICLR)*.